UNIT 4 - TREE

Topic: AVL Imbalance Detection

1. Problem Statement

Write a C++ program to detect LL, RR, LR, and RL imbalances during AVL tree insertion, identify the rotation required, and display the balanced tree.

2. Objective

1. To understand AVL trees and their properties.
2. To detect LL, RR, LR, and RL imbalances during insertion.
3. To perform the appropriate rotations to balance the tree.
4. To display the balanced AVL tree.

3. Introduction

An AVL tree is a self-balancing Binary Search Tree (BST). It maintains balance after insertion by performing rotations when required.

The balance factor of a node is calculated by subtracting the height of its right subtree from the height of its left subtree.

Balance Factor = Height of Left Subtree − Height of Right Subtree

For a balanced AVL tree, each node must have a balance factor of -1, 0, or +1.

4. Types of Imbalance and Rotations

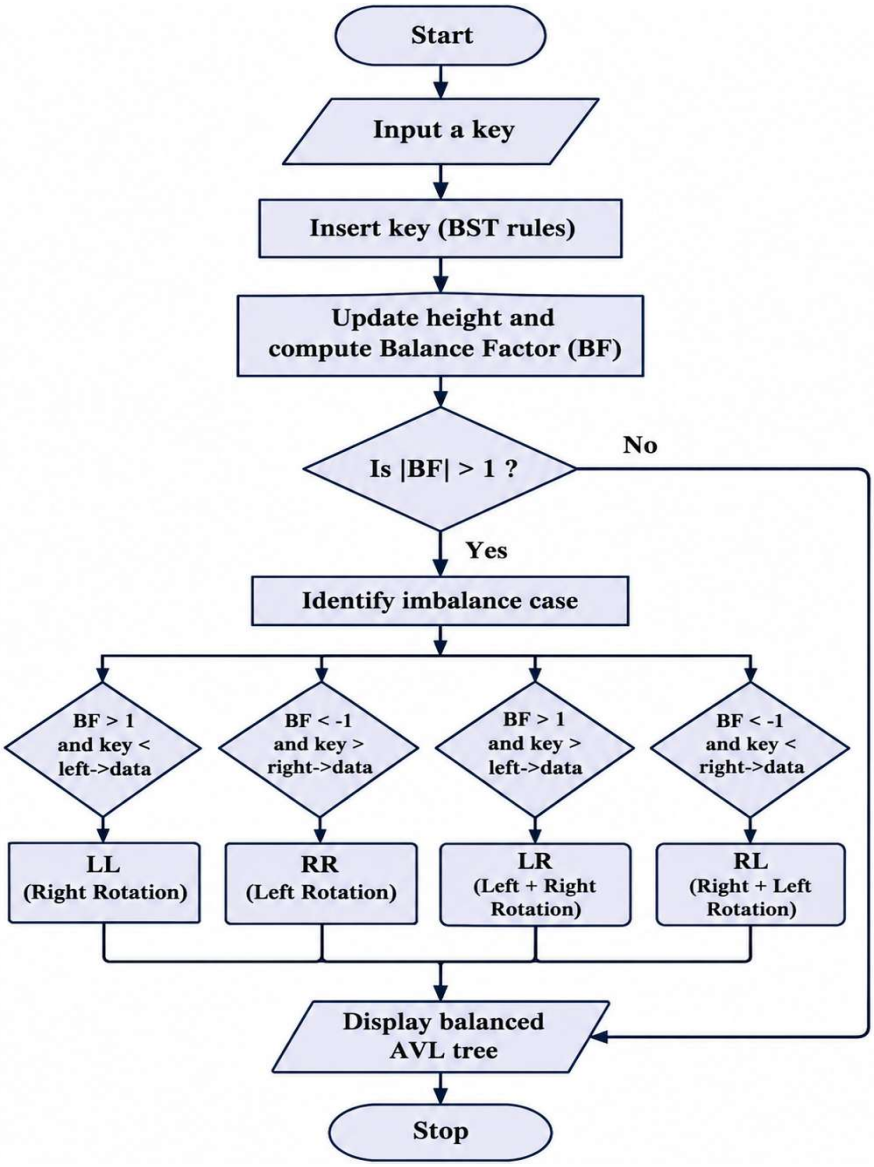
Imbalance	Description	Rotation Required
LL	Left subtree of the left child becomes heavy.	Right Rotation
RR	Right subtree of the right child becomes heavy.	Left Rotation
LR	Right subtree of the left child becomes heavy.	Left Rotation followed by Right Rotation
RL	Left subtree of the right child becomes heavy.	Right Rotation followed by Left Rotation

5. Algorithm

1. Start the program.
2. Create an empty AVL tree.
3. Accept the number of nodes and their values.
4. Insert each value using Binary Search Tree insertion rules.
5. Update the height of each node.
6. Calculate the balance factor.
7. Detect the type of imbalance.
8. Perform the required rotation.

9. Display the inorder traversal and balanced AVL tree.
10. Stop the program.

6. Flowchart



7. Source Code

```
#include <iostream>
#include <algorithm>
using namespace std;

// Structure of an AVL tree node
struct Node
{
   int data;
   Node* left;
```

```
    Node* right;
    int height;
};

// Get height of a node
int height(Node* node)
{
    if (node == NULL)
        return 0;

    return node->height;
}

// Calculate balance factor
int getBalance(Node* node)
{
    if (node == NULL)
        return 0;

    return height(node->left) - height(node->right);
}

// Create a new node
Node* createNode(int value)
{
    Node* node = new Node();

    node->data = value;
    node->left = NULL;
    node->right = NULL;
    node->height = 1;

    return node;
}

// Right rotation for LL imbalance
Node* rightRotate(Node* y)
{
```

```
    Node* x = y->left;

    Node* temp = x->right;

    x->right = y;
    y->left = temp;

    y->height = 1 + max(height(y->left), height(y->right));
    x->height = 1 + max(height(x->left), height(x->right));

    return x;
}

// Left rotation for RR imbalance
Node* leftRotate(Node* x)
{
    Node* y = x->right;
    Node* temp = y->left;

    y->left = x;
    x->right = temp;

    x->height = 1 + max(height(x->left), height(x->right));
    y->height = 1 + max(height(y->left), height(y->right));

    return y;
}

// Insert a value into the AVL tree
Node* insert(Node* node, int value)
{
    // Perform normal BST insertion
    if (node == NULL)
        return createNode(value);

    if (value < node->data)
        node->left = insert(node->left, value);
    else if (value > node->data)
```

```
    node->right = insert(node->right, value);
else
    return node;

// Update height
node->height = 1 + max(height(node->left), height(node->right));

// Calculate balance factor
int balance = getBalance(node);

// LL Case
if (balance > 1 && value < node->left->data)
{
    cout << "LL Imbalance detected. Right Rotation performed.\n";
    return rightRotate(node);
}

// RR Case
if (balance < -1 && value > node->right->data)
{
    cout << "RR Imbalance detected. Left Rotation performed.\n";
    return leftRotate(node);
}

// LR Case
if (balance > 1 && value > node->left->data)
{
    cout << "LR Imbalance detected. Left and Right Rotations performed.\n";

    node->left = leftRotate(node->left);
    return rightRotate(node);
}

// RL Case
if (balance < -1 && value < node->right->data)
{
    cout << "RL Imbalance detected. Right and Left Rotations performed.\n";
```

```
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }

    return node;
}

// Display tree using inorder traversal
void displayInorder(Node* root)
{
    if (root != NULL)
    {
        displayInorder(root->left);
        cout << root->data << " ";
        displayInorder(root->right);
    }
}

// Display tree structure
void displayTree(Node* root, int space = 0)
{
    if (root == NULL)
        return;

    space += 5;

    displayTree(root->right, space);

    cout << endl;

    for (int i = 5; i < space; i++)
        cout << " ";

    cout << root->data << "\n";

    displayTree(root->left, space);
}
```

```
// Main function
int main()
{
    Node* root = NULL;
    int n, value;

    cout << "AVL Imbalance Detection\n";
    cout << "Enter number of nodes: ";
    cin >> n;

    if (n <= 0)
    {
        cout << "Please enter a positive number of nodes.\n";
        return 0;
    }

    for (int i = 0; i < n; i++)
    {
        cout << "Enter value: ";
        cin >> value;

        root = insert(root, value);
    }

    cout << "\nInorder Traversal: ";
    displayInorder(root);

    cout << "\n\nBalanced AVL Tree:\n";
    displayTree(root);

    cout << "\n";
    return 0;
}
```

8. Sample Input

```
Enter number of nodes: 3
Enter value: 30
Enter value: 20
```

```
Enter value: 10
```

9. Sample Output

```
AVL Imbalance Detection
Enter number of nodes: 3
Enter value: 30
Enter value: 20
Enter value: 10
LL Imbalance detected. Right Rotation performed.

Inorder Traversal: 10 20 30

Balanced AVL Tree:

     30

20

     10


--------------------------------
Process exited after 271.8 seconds with return value 0
Press any key to continue . . .
```

10. Applications of AVL Trees

1. AVL trees are used for efficient searching and insertion.
2. They are useful in database indexing.
3. They help maintain sorted data.
4. They are used in applications that require fast search operations.

11. Advantages

1. AVL trees maintain balance automatically.
2. Searching is efficient because the tree height remains logarithmic.
3. Insertion and searching take O(log n) time in a balanced AVL tree.
4. Rotations help maintain the balance of the tree.

12. Conclusion

This project demonstrates AVL tree insertion and the detection of LL, RR, LR, and RL imbalances. The program performs the appropriate rotations to maintain balance and displays the resulting AVL tree. Through this project, the concept of self-balancing Binary Search Trees is understood practically using C++.